RoboBIM: A Policy-Gated, Agent-Compiled IFC-to-Navigation Contract for Robot-Ready Indoor Priors

Abstract— **Building Information Models (BIM), encoded in the Industry Foundation Classes (IFC) schema, contain dense geometric and semantic priors for indoor autonomy, yet contemporary IFC-to-robot pipelines extract only their geometric envelope, discarding the affordance, material, and topological-context fields required for robot-ready planning. Conversely, monolithic large-language-model (LLM) ingestion of raw IFC offers no safety envelope: hallucinated topology, fabricated door clearances, or dropped storeys silently corrupt downstream navigation. We present RoboBIM, a staged, policy-gated agentic pipeline that compiles an IFC file into a versioned *navigation contract* – a three-layer graph $\mathcal{G} = (\mathcal{O}, \mathcal{W}, \mathcal{T})$ comprising per-storey occupancy grids ($\mathcal{O}$), a Voronoi-derived waypoint graph with clearance-annotated edges ($\mathcal{W}$), and a room-transition topology ($\mathcal{T}$), enriched with per-room type, hazard, material, traversability, and building-level navigation policies. A deterministic geometric backbone produces the structural baseline; a seven-stage tool-augmented LLM agent (Gemini 3.0 Flash) probes the IFC through sandboxed code execution, tunes extraction hyperparameters under a clamped parameter policy, infers semantics, and gates every structural edit through a *topology regression filter* that rolls back any candidate that reduces baseline connectivity. On four public IFC models spanning residential, mixed-use, and office layouts (14–64 rooms, 2–6 storeys), RoboBIM increases room-type coverage from 51/131 to 89/131 rooms (+74.5%), augments hazard, material, and traversability fields where the deterministic baseline yields only nulls, and preserves connected topology in all runs at a cost of 403 ± 83 s and 269 ± 67 k input tokens per building. The contract is exported to Nav2, Open-RMF, GraphML, and GeoJSON, and accompanied by a deterministic natural-language briefing for grounding on-board robot LLMs without runtime IFC access.**

I. Introduction

Mobile robots deployed in structured indoor environments routinely reconstruct metric-semantic representations from on-board sensing – Kimera [1], Hydra [2], and 3D scene-graph pipelines – even when a high-fidelity digital twin of the deployment site exists as an IFC model authored at design time [3]–[5]. This duplication discards architectural priors that the robot would otherwise expend minutes re-discovering. The dual extreme is equally unsatisfactory: consuming IFC verbatim yields priors that are geometrically complete yet semantically inert. Room function when `IfcSpace.LongName` is blank, floor material when material assignments are attached to `IfcSlab` subassemblies rather than to spaces, and per-robot traversability – the fields that distinguish a *map* from a *robot-ready prior* – are either absent from the schema or distributed across heterogeneous `IfcPropertySet` instances that elude deterministic parsing.

The IFC semantic gap. The canonical IFC-to-graph pipeline (`IfcSpace` extraction, generalized Voronoi diagram (GVD) segmentation, door-waypoint injection, ROS 2 serialization) is well-established [6]–[10]. It reliably yields occupancy grids, a topological room-transition graph, and a waypoint map, but its semantic ceiling is the schema itself.

Monolithic LLM consumption is unsafe. A tempting alternative is to submit the raw IFC to a frontier LLM and request a robot briefing. Prior work demonstrates that LLMs can reason over BIM in principle – MCP4IFC [11], ifcMCP [12], modular MCP-BIM architectures [13], Text2BIM [14], and BIM2RDT [15] – but these systems are tolerant of ambiguity in ways that robot priors are not. A hallucinated corridor, a fabricated door width, or a silently dropped storey can corrupt downstream planning under failure modes that are difficult to detect at runtime. None of these pipelines provide a mechanism that guarantees the LLM-augmented output never degrades the deterministic topological baseline.

Contribution: staged agentic compilation with a regression gate. We frame IFC-to-prior compilation as a two-layer problem. A *deterministic backbone* extracts the structural graph $\mathcal{G}_{\text{static}} = (\mathcal{O}, \mathcal{W}, \mathcal{T})$. A *staged agentic layer* of seven LLM stages thereafter enriches $\mathcal{G}_{\text{static}}$ with the interpretive fields the backbone cannot infer. The agentic layer operates under two declarative policies, `agent_structural_policy=none` and `agent_parameter_policy=fixed`, that forbid topology and out-of-clamp parameter mutation, and every candidate enrichment passes a *topology regression gate* that rolls back any edit reducing connected-transition count below the deterministic baseline. The resulting artifact is a versioned *navigation contract* bundling occupancy, waypoints, topology, semantics, per-field confidence, and full provenance.

Fig. 1 frames the full vision: an offline agentic IFC-to-prior compiler (top) feeding an online dual-agent navigation loop (bottom). This paper concerns the offline compiler; the online loop is sketched in Sec. V and deferred to future work. Concretely, we contribute:

- **(C1)** A three-layer, robot-consumable navigation graph extracted top-down from IFC, with native exports to Nav2, Open-RMF, GraphML, and GeoJSON.
- **(C2)** A staged, tool-augmented LLM agent (seven stages) that tunes backbone hyperparameters within clamped ranges, infers semantic fields, and emits a deterministic natural-language briefing for an on-board robot LLM.
- **(C3)** A *topology regression gate* that audits every agent-proposed structural edit against the deterministic base-

line and rolls back any regression – the first explicit safety envelope for LLM-mediated IFC enrichment to our knowledge.
- **(C4)** An evaluation on four public IFC models showing a +74.5% lift in room-type coverage (51/131 $\rightarrow$ 89/131), full population of hazard, material, traversability, and policy fields where the baseline reports nulls, and zero regressions of the connected topology.

II. Related Work

Deterministic IFC-to-robot priors. Hamieh et al. [3], [16] extract IFC into weighted hypergraphs for cascaded A*; Follini et al. [4] established the canonical IFC-to-costmap pipeline; subsequent work [5], [10], [17] automated waypoint extraction for construction robots. The IFC-graph literature [6]–[9] formalizes topological graph generation from IFC. All of these methods share a common ceiling: the semantics they expose are exactly those explicit in the IFC schema. RoboBIM's deterministic backbone resides in this lineage; the contribution is the agentic layer that compiles over it.

3D scene graphs and perception-based semantic priors. SayPlan [18], EmbodiedRAG [19], ContextMatters [20], and Taskography [21] construct hierarchical 3D scene graphs from on-board sensing and apply LLMs for high-level task planning. Ray et al. [22] specify structured interfaces for scene-graph reasoning. These pipelines solve the *complementary* problem – semantic richness without an a-priori map. RoboBIM solves the *dual* – semantic richness *from* an a-priori map, with no perception required at compile time. Pix2G [23] bridges the two by pairing IFC with online scene-graph construction; we target the upstream stage of making the IFC itself semantically rich enough to seed such a bridge.

LLM–BIM integration (design-centric). MCP4IFC [11], ifcMCP [12], modular MCP-BIM architectures [13], and Text2BIM [14] confirm that LLMs can interact with IFC via tool-calling interfaces, but target architectural design or generative authoring. BIM2RDT [15] applies agentic reasoning to point-cloud-to-BIM alignment. Kim et al. [24] use LLMs for construction-robot task planning *downstream* of a BIM. None expose a policy-gated, safety-constrained pipeline for compiling IFC into a robot navigation prior; this is the niche RoboBIM occupies.

LLMs for affordance- and capability-aware planning. SayCan [25], AutoGPT+P [26], VAMOS [27], TIC-VLA [28], AINav [29], BrainBody-LLM [30], and IndoorR2X [31] ground LLM plans in robot affordances and adaptive replanning. All assume the semantic prior is *given*. RoboBIM's navigation contract is engineered as a drop-in input to such planners.

Positioning. Table I summarizes the landscape. The distinguishing capabilities of RoboBIM are (i) *staged* rather than monolithic LLM invocation, (ii) an *explicit topology regression gate*, and (iii) a *multi-consumer contract* output rather than a single bespoke serialization.

TABLE I: Positioning. RoboBIM targets agentic IFC-to-prior *compilation* with safety gates, orthogonal to both deterministic IFC pipelines and perception-based semantic priors.

Capability	Det. IFC	3DSG + LLM	RoboBIM
IFC as prior source	✓	✗	✓
Rich semantic annotation	✗	✓	✓
No perception required	✓	✗	✓
Staged LLM with bounded roles	✗	partial	✓
Topology regression gate	✗	✗	✓
Multi-consumer contract	partial	✗	✓
Per-robot traversability	✗	partial	✓

III. Methodology

The RoboBIM compiler comprises a deterministic geometric backbone and a tool-augmented LLM agent wrapper (Fig. 2). The backbone consumes a single IFC file and produces $\mathcal{G}_{\text{static}} = (\mathcal{O}, \mathcal{W}, \mathcal{T})$ – per-storey occupancy grids $\mathcal{O}$, a clearance-annotated waypoint graph $\mathcal{W}$, and a room-transition topology $\mathcal{T}$. The agent wrapper executes seven stages over the same backbone, probing the IFC through a sandboxed Python interpreter, tuning backbone hyperparameters within clamped ranges, and writing semantic fields Σ (room types, hazards, materials, traversability, building-level policies) into the graph. The output $\mathcal{G}_{\text{agent}} = (\mathcal{O}, \mathcal{W}, \mathcal{T}, \Sigma)$ is exported to Nav2, Open-RMF, GraphML, and GeoJSON, alongside a deterministic natural-language briefing for grounding on-board robot LLMs. Per-stage specifications, prompts, clamp ranges, and confidence-threshold tables appear in Appendix .

A. Deterministic backbone

The backbone is a five-module pipeline: **(a) loader** (storey, door, and `IfcSpace` extraction); **(b) occupancy** (per-storey rasterization of walls and columns into a 5 cm grid); **(c) segmentation** (Euclidean signed distance field, GVD skeleton, door-aware connected-component room labelling, with a purely geometric fallback when no `IfcSpace` entities are present); **(d) waypoints** (skeleton tracing into a sparse graph with door-crossing waypoints injected to align the graph with the building's access topology); and **(e) topology** (room-transition graph construction with material attachment from property sets and a cross-storey consistency check that flags stair or elevator transitions whose endpoints fail to align between floors). The output $\mathcal{G}_{\text{static}}$ is produced without LLM intervention and serves as the safety baseline.

B. Agentic enrichment layer

The agent wrapper adds seven stages, each reading from and writing to a shared *Pipeline Context*. **Stage 0** explores the IFC through a sandboxed `IfcOpenShell` [32]-backed Python tool with hard timeout, no file writes, and disallowed imports. **Stage 1** emits clamped extraction hyperparameters (grid resolution, dilation radius, minimum room area, door search radius, include-openings flag) under a parameter policy $\in$ {apply, suggest, fixed}; default is `fixed`. **Stage 2** infers room types: a regex-and-property-set prefilter resolves deterministic cases; remaining rooms on a given storey are

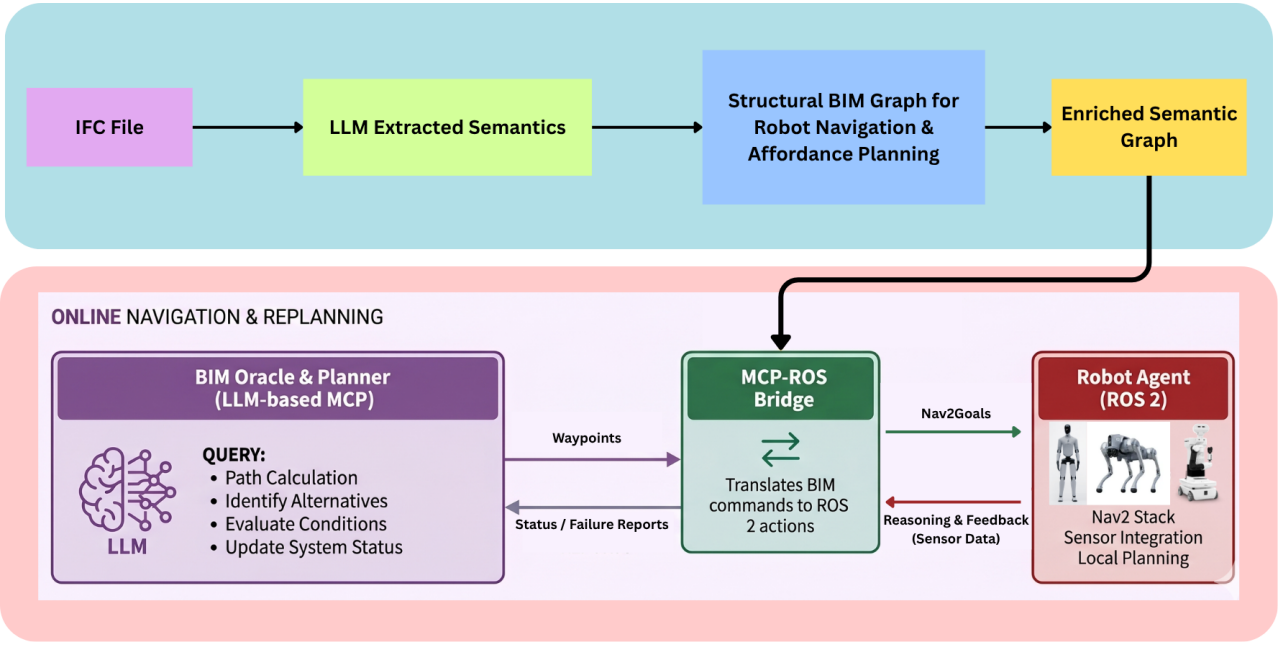


Fig. 1: RoboBIM system vision. **Top (this paper):** IFC is compiled into a multi-layer navigation contract by a staged LLM agent (Gemini 3 Flash), with a deterministic structural backbone and a policy-gated semantic-enrichment layer. **Bottom (outlook, Sec. V):** the contract becomes the substrate for an online BIM-Oracle planner that communicates with a ROS 2 robot agent via an MCP bridge for failure-driven replanning. Contributions in this paper concern the offline agentic prior.

submitted in a single batched prompt to expose cross-room context (an unlabelled space with multiple door transitions bordered by office-typed rooms is reliably inferred as a corridor – a regularity that resists rule-based encoding but is straightforward in-context). **Stage 2b** reviews segmentation and may issue merge directives (confidence ≥ 0.70 applied; splits recorded only). **Stage 3** validates connectivity and emits add/remove/reclassify edits; the topology regression gate audits the resulting connected-transition count and rolls back any edit reducing it below baseline. **Stage 4** writes hazards (controlled vocabulary {wet, obstacle-dense, high-traffic, narrow}), floor materials, and a traversability score in $[0, 1]$. **Stage 5** produces a deterministic natural-language audit and briefing intended to be pasted verbatim into the system prompt of an on-board robot LLM, eliminating runtime IFC access.

C. Output graph

$\mathcal{G}_{\text{agent}}$ has three linked layers (visualised on `BasicHouse` in Appendix Fig. 3). Layer 1 is a per-storey 5 cm occupancy grid with {free, occupied, unknown} cells. Layer 2 is a per-storey GVD-derived waypoint graph in which every edge carries a clearance value $c \in \mathbb{R}_{\geq 0}$ that determines whether a robot of inflated radius r can traverse it ($c \geq r$). Layer 3 is the topological graph of rooms and transitions (doors, wall openings, stair/elevator landings). Cross-layer links bind every room to its waypoint cluster and every transition to its crossing waypoint, supporting hierarchical descent from a topological plan to a metric trajectory. The deterministic baseline reserves but leaves empty the per-room semantic fields (type label with confidence and source, hazard list, floor-material string, traversability $\in [0, 1]$); the agentic

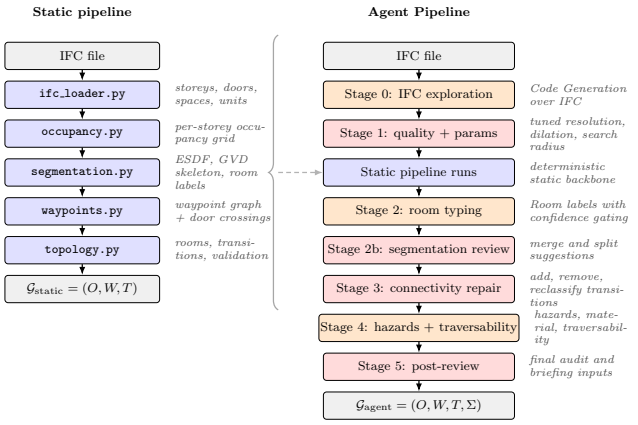


Fig. 2: Static pipeline (left) vs RoboBIM agent pipeline (right). The static pipeline is five deterministic Python modules that produce $\mathcal{G}_{\text{static}}$. The agent pipeline wraps the same backbone with seven LLM stages and a topology regression gate, producing $\mathcal{G}_{\text{agent}} = (\mathcal{O}, \mathcal{W}, \mathcal{T}, \Sigma)$.

layer populates them, and additionally derives building-level policies (tightest door-width invariant, non-navigable storeys) attached to the graph rather than to any single room.

IV. Experiments

A. Setup

We evaluate on four public IFC models – `4351`, `BasicHouse`, `digital_hub`, and `samuel_macalister_sample_house` – covering residential, mixed-use, and office layouts of 14–64 rooms across 2–6 storeys. `BasicHouse` carries no `IfcSpace` entities and exercises the geometric fallback. Provenance,

TABLE II: Per-building results on four IFC models. The *Geometry* columns are identical for both pipelines (the gate forbids regression) and listed once. *Semantic enrichment* columns are written static → agent.

Model	Geometry (preserved) storeys	rooms	conn. trans.	waypoints	Semantic enrichment (static → agent) type	w/ hazard	traversability	material	policy	Agent cost time (s)	tok. in/out
4351	5	22	10	90	1→1	0→3	0→22	0→5	0→5	294	285k/32k
BasicHouse	2	14	6	32	0→7	0→7	0→14	0→7	0→6	489	200k/24k
digital_hub	3	64	60	304	38→64	0→0	0→64	0→0	0→1	423	343k/32k
macalister	6	31	9	392	12→17	0→9	0→31	0→17	0→6	407	348k/41k
total	**16**	**131**	**85**	**818**	**51→89**	**0→19**	**0→131**	**0→29**	**0→18**	1612	1176k/129k

licenses, and raw element counts appear in Appendix G. We run both the deterministic backbone and the full agent pipeline (Gemini 3.0 Flash preview) on every building. Each reported number is a single-run measurement; we discuss this limitation in Sec. V. Comparisons span three axes: the geometric graph, the populated semantic fields, and the wall-clock and token cost of the agent pass.

B. Results

Table II summarises the per-IFC results. Aggregating across the corpus, room-type coverage rises from 51/131 to 89/131 rooms (+74.5%); hazard, traversability, material, and policy fields move from uniformly null in the deterministic baseline to populated in the agentic output. On `BasicHouse`, where no `IfcSpace` entities exist, the agent labels all 7 navigable rooms from geometry and adjacency alone, including hazard inference for kitchen, bathroom, and laundry zones. On `digital_hub`, the agent closes the 38-of-64 gap left by the regex prefilter and reaches full coverage. **No agent-proposed edit reduced connected-transition count below the deterministic baseline on any of the four runs**, confirming that the topology regression gate held in this corpus.

Agent wall-clock time ranges 294–489 s/IFC (mean 403 s); input tokens 200–348 k; output tokens 24–41 k. Per-storey batching keeps the total LLM call count between 7 and 15 per run. Stage 0 and Stage 2 dominate the time budget; Stage 2b dominates on `BasicHouse` (geometric fallback); Stage 3 dominates on `digital_hub` (densest connectivity graph). Per-stage breakdowns appear in Appendix G (Fig. 5, Tables V).

C. Qualitative analysis and online query agent (motivation)

A hero qualitative comparison on Floor 0 of `BasicHouse` (Appendix Fig. 4) contrasts the static and agent outputs: the static pipeline emits uniformly grey rooms with geometric indices; the agent output colours rooms by inferred type (`living_room`, `bedroom`, `bathroom`, `laundry`, `storage`) and surfaces hazard codes (W=wet, O=obstacle-dense, T=high-traffic, N=narrow). Listing 1 (appendix) reproduces a four-room excerpt of the generated briefing – type, hazards, material, traversability, and derived navigation policies (tightest door 0.91 m, robot width limit 0.81 m, caution rooms, non-navigable storeys) – none of which the deterministic baseline populates. The exports and briefing can be served to a robot-side LLM through the Model Context Protocol [33]: when the local planner reports a blocked door or occluded corridor, an on-board agent can query alternatives that satisfy width, hazard, and traversability constraints, rerouting against the prior rather than re-exploring the global state space. Empirical evaluation of this online loop is deferred to future work.

V. Conclusion and Future Work

We presented **RoboBIM**, an IFC-to-navigation-graph compiler that treats IFC processing as semantic enrichment over a deterministic geometric backbone gated by a topology regression filter. On four public buildings the agent raises type coverage by +74.5%, populates hazard, material, traversability, and policy fields where the baseline is null, and preserves connected topology in every run. **Limitations.** Reported numbers are single-run measurements and the regression gate audits a single scalar (connected-transition count); a candidate edit that preserves count but rewires a corridor through a kitchen would slip through. Confidence thresholds (0.70/0.75/0.80) are chosen heuristically. **Future work** addresses each limitation: (i) closing the as-designed/as-built gap through perception fusion, using $\mathcal{G}_{\text{agent}}$ as the stable reference target that Hydra-class [2] systems reconstruct, updated with DAAAM-style [34] captions; (ii) extending the contract with containment and boundary layers from `IfcRelContainedInSpatialStructure` and `IfcRelAssociatesMaterial`; (iii) adversarial probing of the regression gate with synthetic bad-merge prompts; and (iv) end-to-end evaluation of the online query agent sketched in Fig. 1 on real ROS 2 deployments. The contract framing – versioned, multi-consumer, provenance-bearing – directly answers the workshop call for robot-ready geometric and semantic priors that scale beyond bespoke per-deployment integrations.

References

[1] A. Rosinol, M. Abate, Y. Chang, and L. Carlone, "Kimera: From SLAM to spatial perception with 3D dynamic scene graphs," in *International Journal of Robotics Research*, 2021.

[2] N. Hughes, Y. Chang, and L. Carlone, "Hydra: A real-time spatial perception system for 3D scene graph construction and optimization," in *RSS*, 2024.

[3] A. Hamieh, A. A.-R. Moughlbay, B. Marche, and C. Francois, "Semantic navigation using building information on construction sites," *arXiv preprint arXiv:2104.10296*, 2021.

[4] C. Follini, M. Terzer, C. Marcher, A. Giusti, and D. T. Matt, "BIM-integrated collaborative robotics for application in building construction," in *Proceedings of the International Symposium on Automation and Robotics in Construction*, 2020.

[5] K. Kim and M. Peavy, "BIM-based semantic building world modeling for robot task planning," *Automation in Construction*, vol. 138, 2022.

[6] T.-A. Teo and K.-H. Cho, "i-GIT: Intelligent generation of indoor topology from IFC models," *ScienceDirect*, 2018.
[7] Y.-H. Lin *et al.*, "Generating straight skeleton-based navigation networks with IFC," *ScienceDirect*, 2020.
[8] J. Zhu *et al.*, "A new graph-based approach for building information querying," in *CIB W78 Conference*, 2024.
[9] "Semantics-based connectivity graph for indoor pathfinding powered by IFC-Graph," *ScienceDirect*, 2025.
[10] Construction Robotics Authors, "Improving autonomous robotic navigation using IFC files," *Construction Robotics*, 2023.
[11] B. K. Nithyanantham *et al.*, "MCP4IFC: IFC-based building design using LLMs," *arXiv preprint arXiv:2511.05533*, 2025.
[12] J.-R. Lin and P. Pan, "ifcMCP: Enabling LLM agents to talk with IFC files," 2025.
[13] "A modular reference architecture for MCP-servers for agentic BIM interaction," *arXiv preprint arXiv:2601.00809*, 2025.
[14] C. Du *et al.*, "Text2BIM: Generating building models using a LLM-based multi-agent framework," *arXiv preprint arXiv:2408.08054*, 2024.
[15] "BIM2RDT: Building information models to robot-ready site digital twins," *arXiv preprint arXiv:2509.20705*, 2025.
[16] A. Hamieh, A. A.-R. Moughlbay, and B. Marche, "Intuitive BIM-aided robotic navigation and assets localization," *Frontiers in Robotics and AI*, 2025.
[17] S. Kim, H. A. Pham, J. Jang, and G. Lee, "Development of BIM-integrated construction robot task planning system," *Automation in Construction*, 2021.
[18] K. Rana, J. Haviland, S. Garg, J. Abou-Chakra, I. Reid, and N. Suenderhauf, "SayPlan: Grounding large language models using 3D scene graphs for scalable robot task planning," in *CoRL*, PMLR 229, 2023.
[19] M. Booker, C. Agia, P. Sermanet, and A. Z. Ren, "EmbodiedRAG: Dynamic 3D scene graph retrieval for efficient robot task planning," *arXiv preprint arXiv:2410.23968*, 2024.
[20] "ContextMatters: Relaxing goals with LLMs for feasible 3D scene planning," *arXiv*, 2026.
[21] C. Agia *et al.*, "Taskography: Evaluating robot task planning over large 3D scene graphs," in *CoRL*, PMLR, 2022.
[22] A. Ray *et al.*, "Structured interfaces for automated reasoning with 3D scene graphs," *arXiv preprint arXiv:2510.16643*, 2025.
[23] A. Longo *et al.*, "Pixels-to-graph: Real-time integration of BIM and scene graphs," *arXiv preprint arXiv:2506.22593*, 2025.
[24] K. Kim *et al.*, "Framework for LLM-enabled construction robot task planning," *Robotics*, vol. 14, no. 9, p. 117, 2025.
[25] M. Ahn, A. Brohan, N. Brown, *et al.*, "Do as I can, not as I say: Grounding language in robotic affordances," *arXiv preprint arXiv:2204.01691*, 2022.
[26] T. Birr *et al.*, "AutoGPT+P: Affordance-based task planning using LLMs," in *RSS*, 2024.
[27] "VAMOS: A hierarchical VLA model for capability-modulated navigation," *arXiv preprint arXiv:2510.20818*, 2025.
[28] Z. Huang *et al.*, "TIC-VLA: A think-in-control VLA model for robot navigation," *arXiv preprint arXiv:2602.02459*, 2026.
[29] "AINav: Interactive navigation via LLM-based skill tree and adaptive replanning," *arXiv preprint arXiv:2503.22942*, 2025.
[30] "BrainBody-LLM: Grounding LLMs for robot task planning with closed-loop feedback," *Advanced Robotics Research*, 2025.
[31] "IndoorR2X: Indoor robot-to-everything coordination with LLM-driven planning," *arXiv preprint arXiv:2603.20182*, 2026.
[32] IfcOpenShell Contributors, "IfcOpenShell: An open-source toolkit for IFC model parsing and manipulation." `https://ifcopenshell.org`, 2024.
[33] Anthropic, "Model context protocol (MCP) specification." `https://modelcontextprotocol.io`, 2024.
[34] N. Gorlo *et al.*, "DAAAM: Dense annotated affordance maps for robot navigation via vision-language models," *arXiv preprint*, 2025.

APPENDIX

This appendix expands the seven agent stages introduced in Section III. For each stage we describe the inputs it reads, the output it produces, and the conditions under which its proposals are written back into the graph.

A. Stage 0: building exploration

The agent receives a compact summary of the input building file that lists entity counts, storeys, doors, and representative property sets. It is also given a sandboxed Python runner with read-only access to the loaded building model through IfcOpenShell [32]. The runner enforces a hard timeout and blocks imports of non-standard libraries and file writes. The agent responds with free-form findings, which are parsed into evidence items and stored in the Pipeline Context.

B. Stage 1: quality analysis and parameter tuning

The agent reads the building summary together with the Stage 0 findings. It returns a structured object with a completeness score, a recommended strategy, and five numeric parameters: grid resolution, dilation radius, minimum room area, door search radius, and an include-openings flag. All numeric parameters are clamped to safe ranges. A parameter policy controls the write-back behaviour. In the apply mode, the parameters are used directly. In the suggest mode, they are used only if they lie inside the clamp range. In the fixed mode, which is the default, they are logged but not used.

C. Stage 2: room-type inference

A regex prefilter types rooms with obvious names, such as corridor, CX, or WC. An element-set prefilter handles rooms with matching content – a room with two or more sanitary terminals is treated as a bathroom. Rooms resolved by either prefilter skip the LLM. The remaining rooms on a given storey are sent to the LLM in a single batched prompt. A newly assigned type requires a confidence of at least 0.80, and overriding an existing type requires a confidence of at least 0.75.

D. Stage 2b: segmentation review

The agent reads the room list and the transitions, then proposes merge or split directives. Merges with a confidence of at least 0.70 are applied at the topology layer: the bounding boxes are combined, the transitions are reassigned, and the absorbed room is removed. Split directives are recorded only, because they require label-map access that is not preserved after extraction.

E. Stage 3: connectivity validation and repair

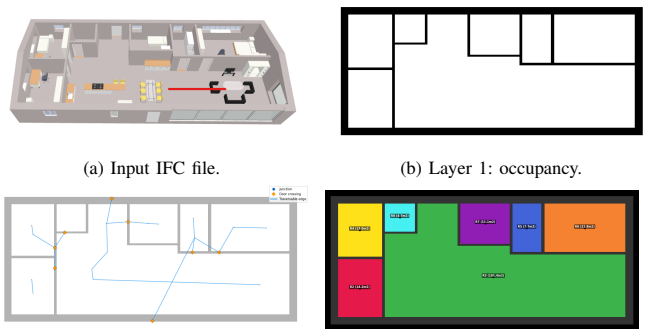
The agent produces a list of issues, such as isolated rooms or suspicious connections. It then outputs a repair plan with three directive kinds: `add_connection`, `remove_connection`, and `reclassify`. Directives with a confidence of at least 0.70 are applied. The topology regression gate then checks the resulting connected-transition count, and any edit that reduces it is rolled back.

F. Stage 4: hazards and traversability

For each room, the agent writes a list of hazard flags from a controlled vocabulary, a floor-material string, a traversability score in $[0, 1]$, a short robot-obstacles string, and a prose note. The stage reuses the prefilter and the per-storey batching introduced in Stage 2.

G. Stage 5: post-extraction review

The agent reads the final enriched graph and returns a short audit that contains an overall quality score, a completeness score, a gaps list, and improvement suggestions. The audit is stored in the Pipeline Context and is used as input to the natural-language briefing.



(a) Input IFC file. (b) Layer 1: occupancy. (c) Layer 2: waypoints. (d) Layer 3: rooms.

Fig. 3: Input IFC file and the three linked layers on `BasicHouse`, ground floor. Top-left: input IFC rendering. Top-right: Layer 1, 5 cm occupancy grid. Bottom-left: Layer 2, GVD-derived waypoint graph with clearance-annotated edges. Bottom-right: Layer 3, room-and-transition topology. Cross-layer links (not drawn) connect each room to its waypoint cluster and each door to its crossing waypoint.

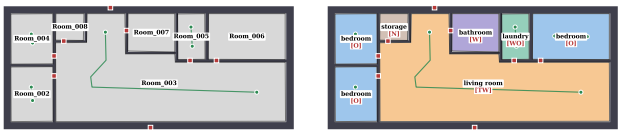


Fig. 4: Floor 0 in `BasicHouse`. Left: static output. Right: agent output. Rooms coloured by inferred type with hazard codes in brackets (W=wet, O=obstacle-dense, T=high traffic, N=narrow).

The four building models used in the main evaluation are all public and redistributable, and each is taken directly from its original repository. `4351`[1] and `samuel_macalister_sample_house`[2] come from the BIMserver TestFiles collection. `digital_hub`[3] comes from the RWTH E3D DigitalHub release. `BasicHouse`[4] comes from the BIM-Whale samples repository. Together, these sources span several design disciplines and both major revisions of the IFC file format.

Table III lists the upstream origin, the license, and the intended building use for each model. Table IV lists raw statistics extracted from each file: the schema version, the on-disk file size, and the total counts of storeys, doors, rooms, and walls. These counts are independent of our pipeline and reflect the authoring state of each file.

`digital_hub` is the densest building in the corpus, with 64 annotated rooms and 64 doors on three storeys,

[1] https://github.com/opensourceBIM/TestFiles
[2] https://github.com/opensourceBIM/TestFiles
[3] https://github.com/RWTH-E3D/DigitalHub
[4] https://github.com/andrewisen/bim-whale-ifc-samples/tree/master/BasicHouse

TABLE III: Provenance of each building model in the corpus.

Model	Upstream source	License	Use
4351	BIMserver TestFiles	GPL v3	office
BasicHouse	BIM-Whale Samples	MIT	residential
digital_hub	RWTH E3D DigitalHub	MIT	mixed-use
macalister	BIMserver TestFiles	GPL v3	residential

TABLE IV: Raw building-file statistics extracted from each model. The rooms column shows how many rooms are explicitly annotated by the author; a zero means the pipeline must recover rooms geometrically.

Model	Schema	Size (MB)	Storeys	Doors	Rooms	Walls
4351	IFC2X3	7.3	5	17	6	43
BasicHouse	IFC2X3	51.7	2	8	0	13
digital_hub	IFC4	9.0	3	64	64	178
macalister	IFC4	25.4	6	16	14	48

and tests the pipeline on a richly annotated model. `4351` is a 5-storey office with only 6 rooms annotated, so the agent must infer room types on a partially annotated file. `BasicHouse` has no room annotations at all, so segmentation falls through to the geometric GVD pass described in Section III; it is also the largest file on disk, because it carries detailed geometry for a small floor plan. `samuel_macalister_sample_house` is a 6-storey residential model with mechanical and plumbing content, and stresses the cross-storey consistency check.

Fig. 5 stacks the seven stage timings per building and overlays the input-token count on a secondary axis. Stage 0 is always the first LLM stage; its time scales with the complexity of the input file, because the agent controls how many code-execution snippets to issue. Stage 2 (room-type inference) scales with the storey count, because it is batched per storey. Stage 2b (segmentation review) is the largest single cost on `BasicHouse`, where the lack of annotated rooms forces a richer geometric review. Stage 3 is the largest cost on `digital_hub`, which has the densest connectivity graph in the corpus.

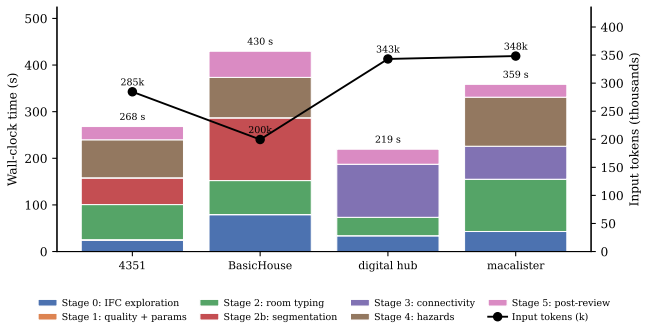


Fig. 5: Per-IFC agent wall-clock time stacked by stage (bars, left axis), with the input-token count overlaid as a black line (right axis). Stage 0, Stage 2, Stage 2b, and Stage 4 dominate the time budget. Confidence-gated per-storey batching keeps the total LLM call count between 7 and 15 per run.

Table V reports the LLM usage per building. Both the call count and the token usage are dominated by Stage 2 and Stage 4 on the smaller buildings. On `BasicHouse`,

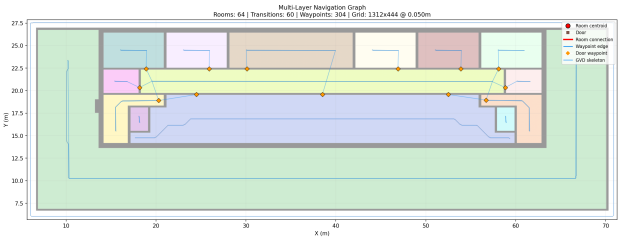
Stage 0 dominates, because the agent chose to issue more exploration snippets than on the other three buildings.

TABLE V: LLM usage per building.

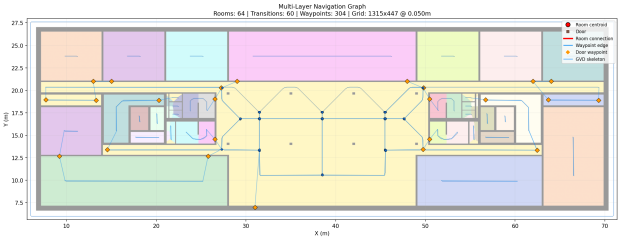
Model	LLM calls	Code execs	Tokens in (k)	Tokens out (k)
4351	13	28	285	32
BasicHouse	9	15	200	24
digital_hub	7	16	343	32
macalister	15	27	348	41
total	44	86	1176	129

This appendix collects additional per-storey visualisations produced by the agent pipeline on two of the four buildings. Each panel is a composite view written by the pipeline that overlays the occupancy grid, the waypoint graph, and the room segmentation on a single storey.

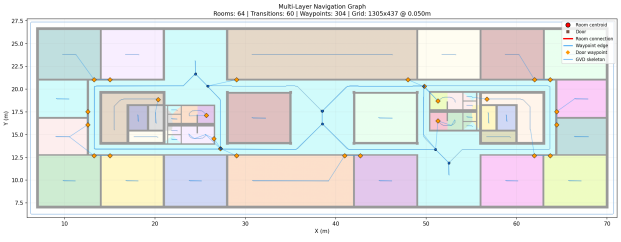
Fig. 6 shows the three storeys of `digital_hub`. The building has annotated rooms on every floor, so the segmentation follows the authored partitions and the waypoint graph densifies cleanly around the shared corridor on each level.



(a) Basement (B01_OKRD).



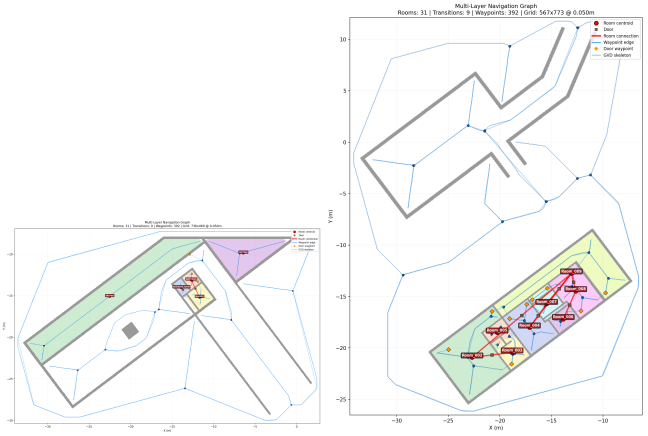
(b) Ground floor (E00_OKRD).



(c) First floor (E01_OKRD).

Fig. 6: Per-storey composite for `digital_hub`. Each panel overlays the occupancy grid (grey), the waypoint graph (blue nodes and edges), and the room segmentation (coloured regions) on one storey of the building.

Fig. 7 shows two of the six storeys in `samuel_macalister_sample_house`. The dwelling has a residential layout that varies across floors, and the waypoint graph contracts around the stair and landing regions on each level.



(a) Level 1. (b) Level 2.

Fig. 7: Composite view on two storeys of `samuel_macalister_sample_house`. The pipeline produces a consistent waypoint graph across levels, with room boundaries drawn from the authored room partitions where available.

Listing 1: Short summary of the BasicHouse briefing derived from the agent trace. The full agent trace is larger and includes stage-by-stage details.

```
Room_003: living_room, concrete, trav=0.80
  hazards: [high_traffic, wet_area]
  notes:   circulation hub, 8 door transitions

Room_005: laundry, concrete, trav=0.50
  hazards: [wet_area, obstacle_dense]
  notes:   narrow channels around appliances

Room_007: bathroom, concrete, trav=0.60
  hazards: [wet_area]
  notes:   constrained layout, plumbing fixtures

Room_008: storage, concrete, trav=0.90
  hazards: [narrow_passage]
  notes:   small walk-in closet

derived_policies:
  tightest_door_m:       0.91
  width_limit_m:         0.81
  caution_rooms:         [Room_003, Room_005, Room_007]
  non_navigable_storeys: [Floor_1]
```